Data Process & Software Modelling

Assessment Phase 1: Project Progress Report and Presentation (50%)

Presented By:

Christian Danielle B. Cantos

Submitted To:

Abubakar Siddique

April 2026

Task 1: Organizational and System Analysis

Overview of the Selected Organization

Air New Zealand is the national flag carrier of New Zealand, headquartered in Auckland and a member of the Star Alliance since 1999. The airline operates an extensive global network that supports both passenger and cargo services across domestic and international routes. Each year, Air New Zealand transports more than sixteen million passengers and operates over four hundred flights daily to forty-nine destinations worldwide.

As a modern airline, Air New Zealand relies heavily on a suite of digital systems that support critical business functions, including booking, ticketing, flight management, customer service, and revenue operations. These systems form the backbone of the airline's operational efficiency and customer experience. The Customer & Digital Department is responsible for overseeing and maintaining these digital platforms, ensuring seamless service delivery across all customer touchpoints.

According to the airline's 2025 annual report, Air New Zealand operates a fleet of 113 aircraft and employs approximately 11,700 staff globally. The scale of its operations underscores the importance of robust, reliable, and secure digital systems to support day-to-day activities and long-term strategic goals.

Departmental structure and responsibilities

Here are the key departments that are relevant to my flight booking system:

1. **Finance & Corporate Services Department** – revenue management, fair pricing, payment processing, refunds, and financial reporting. The booking system handles transactions, fare classes, taxes, and booking revenue tracking.
2. **Operations Department** – manages flight operations such as flight schedules, aircraft availability, gate assignments, and on-ground services. The booking system must reflect real-time flight status, capacity, and operational changes like delays or cancellations.
3. **Customer & Digital Department** – responsible for technology platforms, digital innovation, customer-facing application, and data systems. The booking system is a core digital platform this team designs, builds, and

maintains. It also handles passenger profiles, booking modifications, and booking history.

4. **People & Culture –** manages staff roles, access permissions, and system user management. The booking system needs role-based access control to distinguish agents, admins, and customers within the platform.

Identification of stakeholders and user roles

In this system, there are 6 stakeholders that will be directly and indirectly involved, which not only affects the main users through operations but heavily relies on the system's output.

Stakeholders:

1. **Customers (Passengers) –** they use the system to search & view flight details, book flights, manages their bookings.

2. **Admin –** they use the system to control bookings, such as viewing bookings details, monitor seat availability and updates flights.

3. **Guest –** they use the system to search flights, and view flights availability. They can only book flights when they register.

4. **System Developers** – we as a developer we do not directly use the system but are affected by it. We are the once who build, test and maintain the system.

5. **Airline Management –** they don't use the system directly but highly affected by it. They rely on the system's output for flight operations, revenue, scheduling, and customer experience.

6. **IT Support / System Administrator –** they don't use the system directly but highly affected by it. They are the ones who maintain and protect the system.

The Flight Booking System is going to be used by these 3 key user roles, each with different access levels.

User Roles:

1. **Admin –** manages flight details, update flights, they exist to maintain accuracy and integrity of the flight database.

2. **Customers –** a registered user who can book flight, check flight availability, manage their bookings.

3. **Guest –** a non-registered user who has limited access.

System Scope, Boundaries, and Assumptions

System Scope

These are the features the system will include:

1. **User Authentication**
    - Login (Existing Customer)
    - Register (New Customer)
    - Role Identification (Admin, Customer, Guest)

2. **Flight Management (Admin-only Access)**
    - Add new flights
    - Update existing flights
    - Remove flights
    - View all flights
    - View flight occupancy

3. **Flight Search & Viewing**
    - Search flights by origin, destination, and date
    - View flight details

4. **Booking Management**
    - Book flights

- View bookings
- Cancel bookings

5. **Data Storage**
    - Store users, flights, and bookings
    - Database System (SQL)

6. **Input Validation**
    - Validate flight details, seat availability, and user inputs

System Boundaries

This defines what the system will not do:

1. **Online Payment Processing**
- No credit card, PayPal, or banking integration
- Only simulated booking confirmation

2. **Graphical User Interface**
- Console Interface Only

3. **Seat Map Visualization**
- No graphical seat selection

4. **Round Trip Bookings**
- Only single-flight, one-way bookings

5. **Automatic pricing algorithms**
- No dynamic pricing or revenue management

6. **Email or SMS notifications**
- No external communication services

7. **Real-world security protocols**
- No encryption, hashing, or enterprise-level security

System Assumptions

User & Data Assumptions

- User provides valid and truthful information (name, date of birth, address, phone number).
- Admins are trusted and will not misuse their privileges.
- All users have basic knowledge of console navigation.

Technical Assumptions

- The console runs smoothly without any bugs.
- If used, data files must not be corrupted or altered.
- No multiple users can have access to the system.

Operational Assumptions

- No real-time updates are required (e.g., Flight delays, cancellations)
- The number of flights and bookings is manageable within console limitations.
- Flight data is manually maintained by Admin.

Environment Assumptions

- The system will run in a stable environment without unexpected shutdowns.
- System Administrators will handle**:**
    - Backup
    - Recovery
    - System Maintenance
    - Security Policies

Task 2: Requirements Engineering

Here we list down every functional and non-functional requirement of the flight

Functional Requirements

User Authentication & Account Management

FR1: The system shall allow users to login using valid username and password.

FR2: The system shall validate user credentials against the stored data.

FR3: The system shall determine the user's role after successful login.

FR4: The system shall allow new users to create an account by providing required personal information.

FR5: The system shall prevent registration if the username already exists.

FR6: The system shall allow users to log out and return to the main menu.

Role-Based Access Control

FR7: The system shall display the admin menu when an admin logs in.

FR8: The system shall display the customer menu when a customer logs in

FR9: The system shall display the guest menu when no user is logged in.

FR10: The system shall restrict access to admin-only functions for non-admin users.

Flight Management (Admin only)

FR11: The system shall allow admins to add new flights by entering flight details (flight number, origin, destination, date, time, seat capacity, price).

FR12: The system shall allow admins to update existing flights from the system.

FR13: The system shall allow admins to remove flights from the system.

FR14: The system shall allow admins to view a list of all flights.

FR15: The system shall allow admins to view the number of booked and available seats for each flight.

Flight Search & Viewing

FR16: The system shall allow users to search for flights by origin, destination, and date.

FR17: The system shall allow users to view detailed information about a selected flight.

Booking Management (Customer Only)

FR18: The system shall allow customers to book a seat on a selected flight.

FR19: The system should validate that the number of seats requested does not exceed the available seats.

FR20: The system shall generate a booking record and confirmation code after a successful booking.

FR21: The system shall allow customers to view all their existing bookings.

FR22: The system shall allow customers to cancel their own bookings.

FR23: The system should update the number of seats booked after each booking or cancellation.

FR24: The system shall allow customers to view all their flight history.

Data Storage & Management

FR25: The system shall store user account information in a persistent storage medium.

FR26: The system shall store flight information in a persistent storage medium.

FR27: The system shall store booking information in a persistent storage medium.

FR28: The system shall load all stored data when the application starts.

FR29: The system shall save all updated data when the application closes.

Input Validation

FR30: The system shall validate all user inputs to ensure they are in the correct format.

FR31: The system shall validate that a selected flight exists before processing a booking or update.

FR32: The system shall prevent booking requests with zero, negative, or non-numeric seat values.

System Navigation & User Interface

FR33: The system shall display clear menus for each user role.

FR34: The system shall display meaningful error messages when invalid input is entered.

FR35: The system shall allow users to return to the main menu after completing an action.

Non-Functional Requirements

Performance Requirements

NFR1: The system shall load all required data within 3 seconds on startup.

NFR2: The system shall respond to user inputs within 1 second under normal operating conditions.

NFR3: The system shall process booking operations within 2 seconds.

NFR4: The system shall support up to 1,000 flights and 10,000 bookings without performance degradation.

Reliability Requirements

NFR5: The system shall maintain data integrity during all read/write operations.

NFR6: The system shall not lose data during normal operation.

NFR7: The system shall properly handle invalid inputs without crashing.

NFR8: The system shall recover gracefully from minor errors.

Availability Requirements

NFR9: The system shall be available for use whenever the host machine is operational.

NFR10: The system should not require an internet connection to function.

NFR11: The system shall allow users to resume normal operation after restarting the application.

Usability Requirements

NFR12: The system shall provide clear and readable text-based menus.

NFR13: The system shall display meaningful error messages for invalid inputs.

NFR14: The system shall allow users to return to the main menu from any submenu.

NFR15: The system shall require no prior technical knowledge beyond basic console navigation.

Security Requirements

NFR16: The system shall restrict admin-only functions to authenticated admin users.

NFR17: The system shall store user passwords in a secure format.

NFR18: The system shall prevent unauthorized access by validating all login attempts

NFR19: The system shall ensure that customers can only view and manage their own bookings.

Maintainability Requirements

NFR20: The system shall be modular, separating logic into components

NFR21: The system shall allow new features to be added without major restructuring.

NFR22: The system shall use clear and consistent naming conventions in code.

NFR23: The system shall allow flight, user, and booking data to be updated without modifying core logic.

Portability Requirements

NFR24: The system shall run to any machine that supports a standard console/terminal.

NFR25: The system shall not depend on platform-specific features.

NFR26: The system shall run on Windows, macOS, or Linux without require major changes.

Data Integrity Requirements

NFR27: The system shall prevent overbooking by validating seat availability before confirming a booking.

NFR28: The system shall update seat counts immediately after booking or cancellation.

NFR29: The system shall ensure that flight, user, and booking data remain synchronized across all operations.

Backup & Recovery Requirements

NFR30: The system shall save all data to persistent storage on exit.

NFR31: The system shall load the most recent saved data on startup.

Traceability between stakeholders and requirements

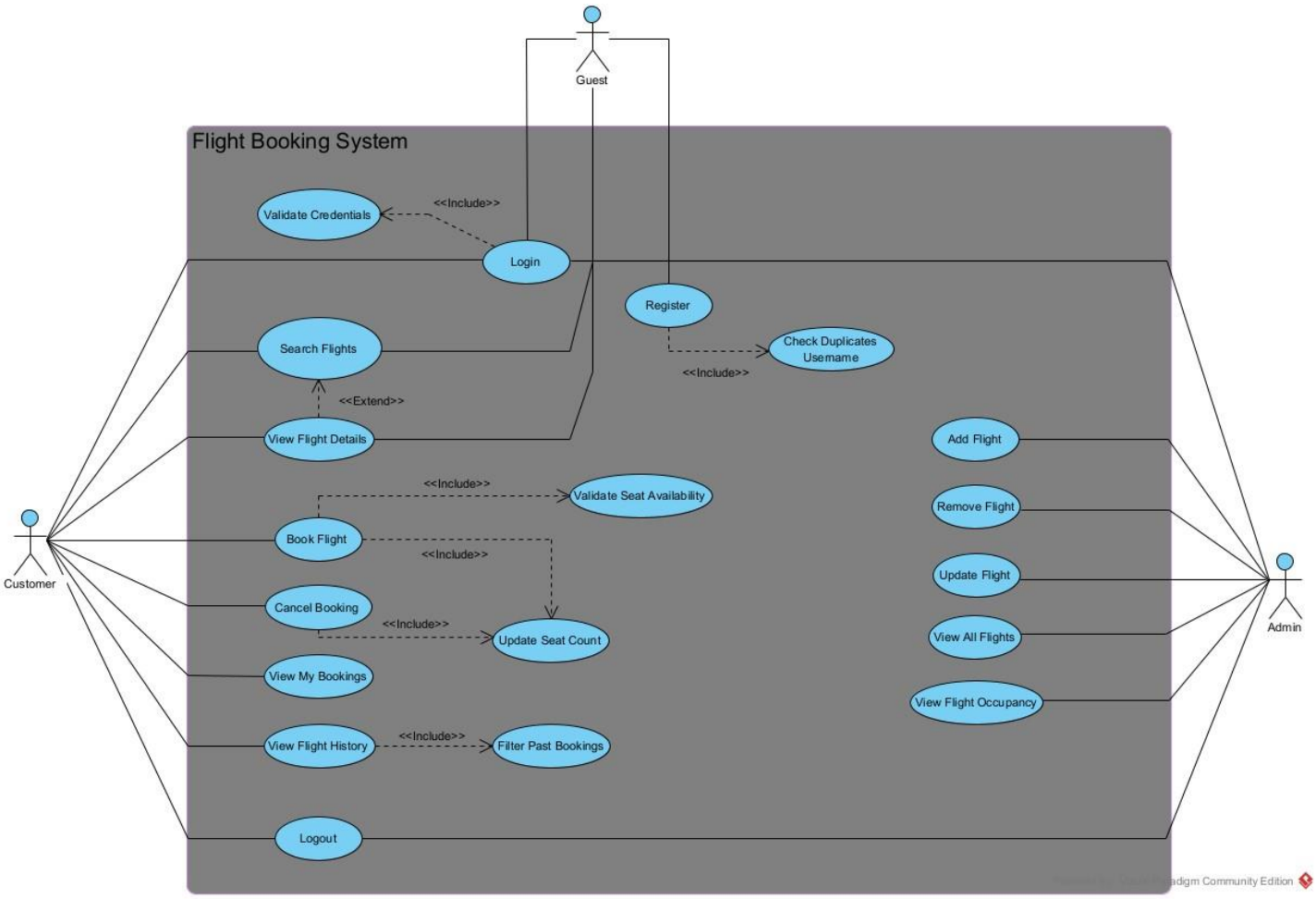
Stakeholders	Functional Requirements
Customers	FR1, FR2, FR3, FR4, FR5, FR6, FR8, FR10, FR16, FR17, FR18, FR19, FR20, FR21, FR22, FR23, FR24, FR25, FR27, FR28, FR29, FR30, FR31, FR32, FR33, FR34, FR35,
Admin	FR1, FR2, FR3, FR6, FR7, FR10, FR11, FR12, FR13, FR14, FR15, FR16, FR17, FR25, FR26, FR28, FR29, FR30, FR31, FR33, FR34, FR35
Guests	FR4, FR9, FR16, FR17, FR28, FR29, FR30, FR33, FR34, FR35
Developers	FR7, FR8, FR9, FR10, FR11, FR12, FR13, FR14, FR15, FR19, FR23, FR25, FR26, FR27, FR28, FR29, FR30, FR31, FR32
Airline Management	FR11, FR12, FR13, FR14, FR15, FR19, FR23, FR26, FR27, FR31, FR32
IT Supports	FR2, FR5, FR10, FR25, FR26, FR27, FR28, FR29, FR30

Stakeholders	Non-Functional Requirements
Customers	NFR1, NFR2, NFR3, NFR4, NFR5, NFR6, NFR7, NFR8, NFR9, NFR10, NFR11, NFR12, NFR13, NFR14, NFR15, NFR16, NFR17, NFR18, NFR19, NFR24, NFR25, NFR26, NFR27, NFR28, NFR29, NFR30, NFR31
Admin	NFR1, NFR2, NFR4, NFR5, NFR6, NFR7, NFR8, NFR9, NFR10, NFR11, NFR12, NFR13, NFR14, NFR15, NFR16, NFR17, NFR18, NFR23, NFR24, NFR25, NFR26, NFR29, NFR30, NFR31
Guests	NFR1, NFR2, NFR4, NFR7, NFR8, NFR9, NFR10, NFR11, NFR12, NFR13, NFR14, NFR15, NFR24, NFR25, NFR26, NFR30, NFR31
Developers	NFR1, NFR2, NFR3, NFR4, NFR5, NFR6, NFR7, NFR8, NFR16, NFR17,

	NFR18, NFR19, NFR20, NFR21, NFR22, NFR23, NFR24, NFR25, NFR26, NFR27, NFR28, NFR29, NFR30, NFR31
Airline Management	NFR4, NFR5, NFR6, NFR23, NFR27, NFR28, NFR29
IT Supports	NFR1, NFR2, NFR3, NFR4, NFR5, NFR6, NFR7, NFR8, NFR9, NFR10, NFR11, NFR16, NFR17, NFR18, NFR19, NFR20, NFR21, NFR23, NFR24, NFR25, NFR26, NFR29, NFR30, NFR31, NFR32

Task 3: UML Requirements Modelling

Use Case Diagram covering full system scope



The use case diagram for the flight booking system illustrates the interactions between three primary actors – Customer, Guest, Admin and the system's core functionalities. Each actor represents a distinct role with different levels of access and responsibilities within the system.

Guests are unregistered users who can search for flights and view flight details, but they must be registered or log in if they want to book a flight. **Customers** are registered users who have access to all booking operations such as search flights, view flight details, booking flights, cancelling booking or even accessing their active bookings and viewing their previous flights. **Admins** have a high access control in the system, they operate flight operations such as adding

flights, removing flights, updating flight information they can also view all existing flights and check flight occupancy.

The diagram uses includes relationships to represent mandatory sub-processes that must occur within certain use cases. *Login* includes *validating credentials* this sub-process makes sure that no one can have access to the system unless they are a registered user. *Register* includes *checking duplicate usernames* so that there will be no inconsistencies in data. *Booking flights* include *validating seat availability,* this checks if the seats are booked or not, this ensures data integrity and it includes *updating seat count*. *Cancel booking* includes *updating seat count* this sub-process makes sure that seat counts are accurate and avoid any double-booking. *View flight history* includes *filter past bookings* this is a sub-process where users can filter out their past bookings while viewing their flight history.

An extend relationships is used between *search flights* and *view flight details* this indicates that viewing flight details is an optional action that may occur after a user performs a flight search.

This use case diagram provides a high-level overview of how different users interact with the system and how internal processes support the main functionalities.

Minimum of 5 fully dressed user case descriptions

ID	**UC01**
Name	Login
Description	Users use their saved credentials to enter the system and to have access to a specific menu
Pre-conditions	Users must be registered in the system
Pre-conditions	System must be running
Event flow	1. Users select "Login" from the main menu 2. System asks for username and password 3. Users enter credentials 4. System validates entered credentials 5. System identifies the user's role (customer/admin) 6. System displays the appropriate menu for the role.
Post-condition	**Success:** User is logged in and session is created **Failure:** User remains logged out
Extension points	None
Triggers	Users must run the system to login.

ID	**UC05**
Name	Book Flight
Description	Customers are allowed to reserve seats on a selected flight
Pre-conditions	Flight must be existing
Pre-conditions	Customers must log in with their accounts.
Event flow	1. System prompt for number of seats 2. Customer enters seat count 3. System performs “Validate Seat Availability” 4. If seats are available, system creates a booking 5. System performs “Update Seat Count” 6. System stores the booking 7. System confirms successful booking.
Post-condition	**Success:** Booking is created and seat count updated. **Failure:** No booking is created
Extension points	None
Triggers	Customers must search for flights.

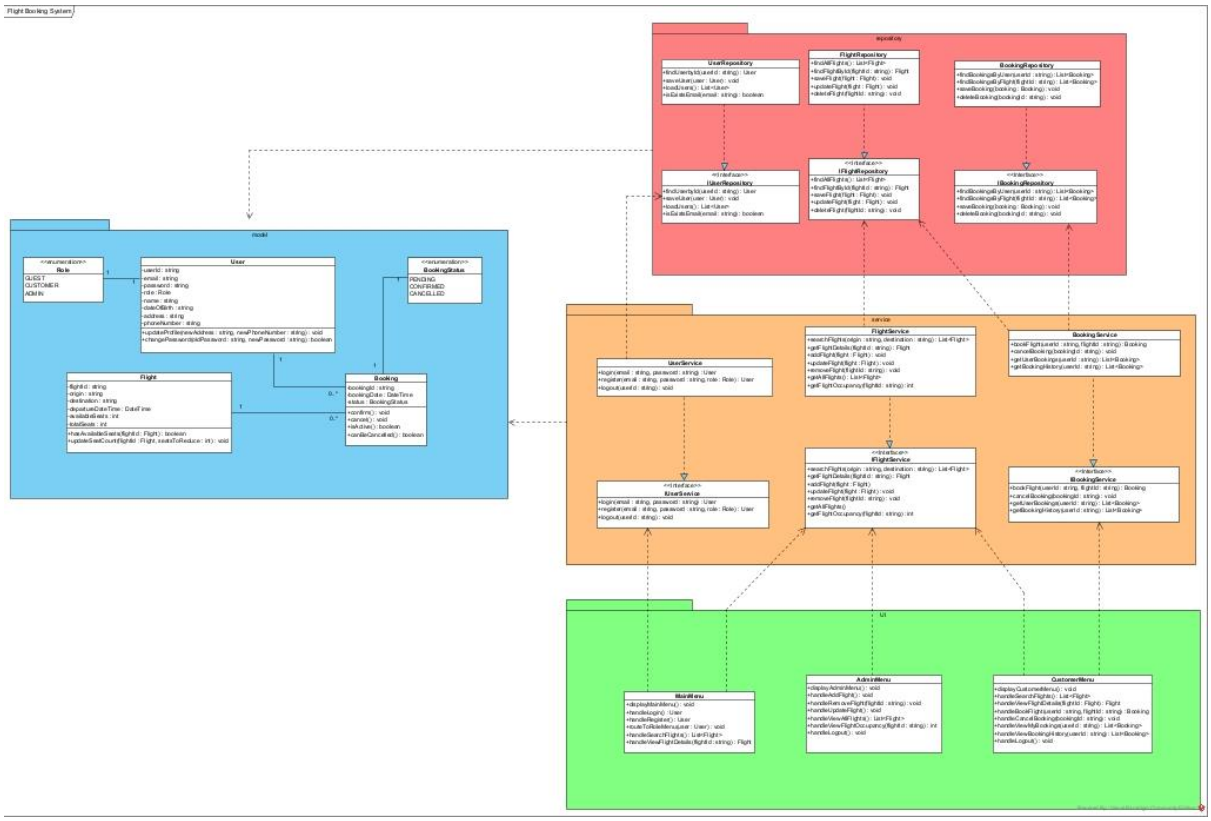
ID	**UC04**
Name	View Flight Details
Description	Users view flight details after searching and entering flight information
Pre-conditions	Flights must exist.
Event flow	1. Users must select “Search Flights” 2. System display search flight prompts 3. Users must enter flight information. 4. System checks if the flight based on the information entered does exists. 5. System Retrieves matching flights 6. System display list of flights. 7. User can optionally “View Flight Details” 8. If yes, system displays flight details.
Post-condition	**Success:** Users can view flight details **Failure:** The flight does not exist.
Extension points	Use case 3 “Search Flights”
Triggers	Users must enter flight information.

ID	**UC02**
Name	Register
Description	Users must be registered to have access to a role-specific menu.
Pre-conditions	Users must enter username and password to create an account
Event flow	1. The system displays main menu 2. Users selects “Register” 3. The system asks for username and password 4. The user must enter credentials. 5. The system checks if the username already exists. 6. The system creates the account and saves it.
Post-condition	**Success:** The system created a new account and saved it. **Failure:** User remains Guest.
Extension points	None
Triggers	User must enter username and password

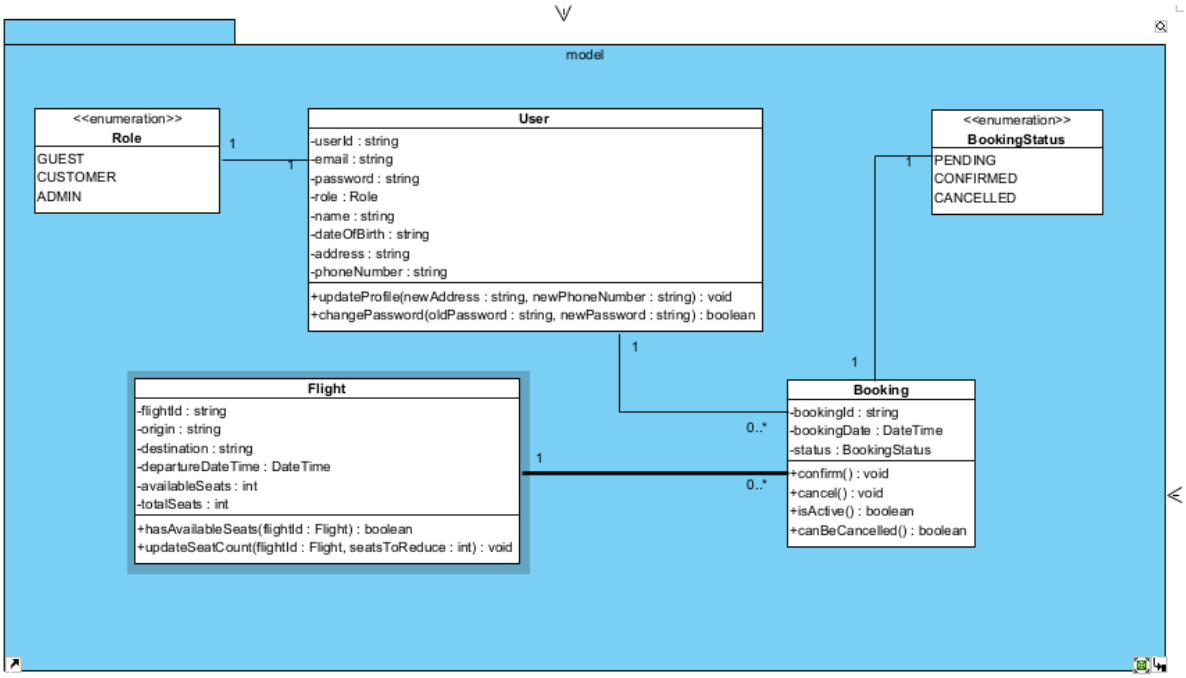
ID	**UC03**
Name	Search Flights
Description	Users search flights based on flight information
Pre-conditions	Users must enter flight information (date, time, origin, destination)
Event flow	1. Users must select "Search Flights" 2. System prompts for search criteria 3. Users must enter flight information. 4. System checks if the flight based on the information entered does exists. 5. System retrieves matching flights 6. System display list of flights 7. User may optionally choose "View Flight Details"
Post-condition	**Success:** Users can pick from the list of flights displayed **Failure:** There are no existing flights based on the flight information given by the user.
Extension points	Use Case 4 "View Flight Details"
Triggers	Users must enter existing flight information.

Task 4: Initial Structural Design and Implementation Planning

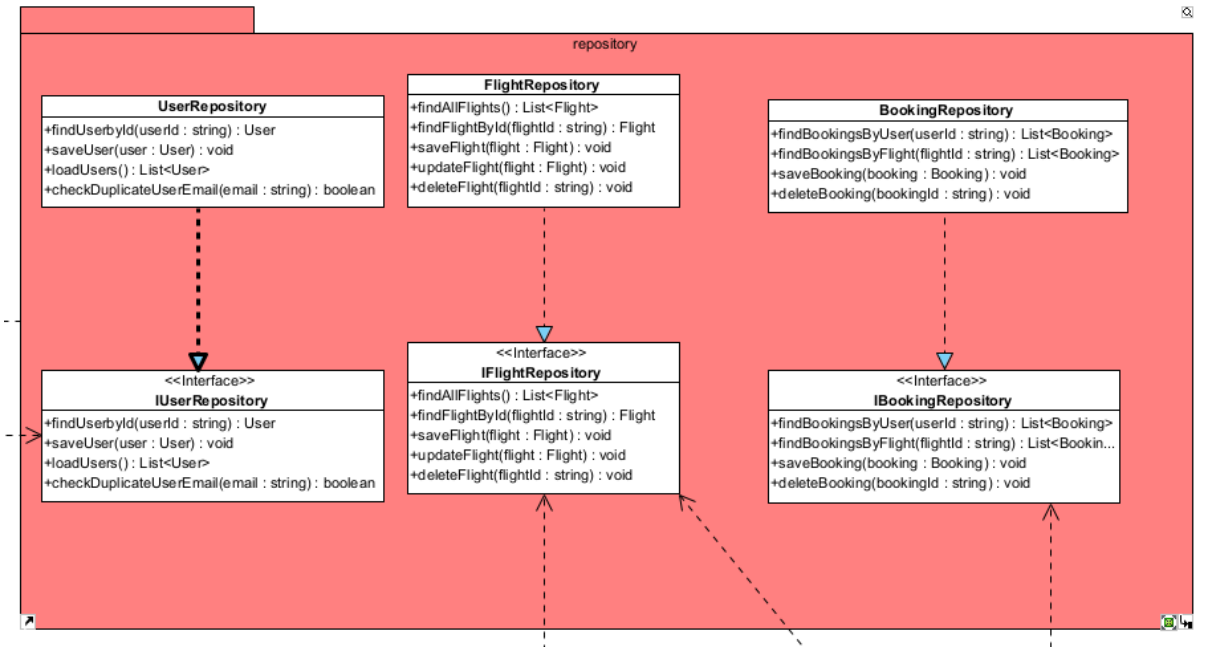
Initial UML Class Diagram



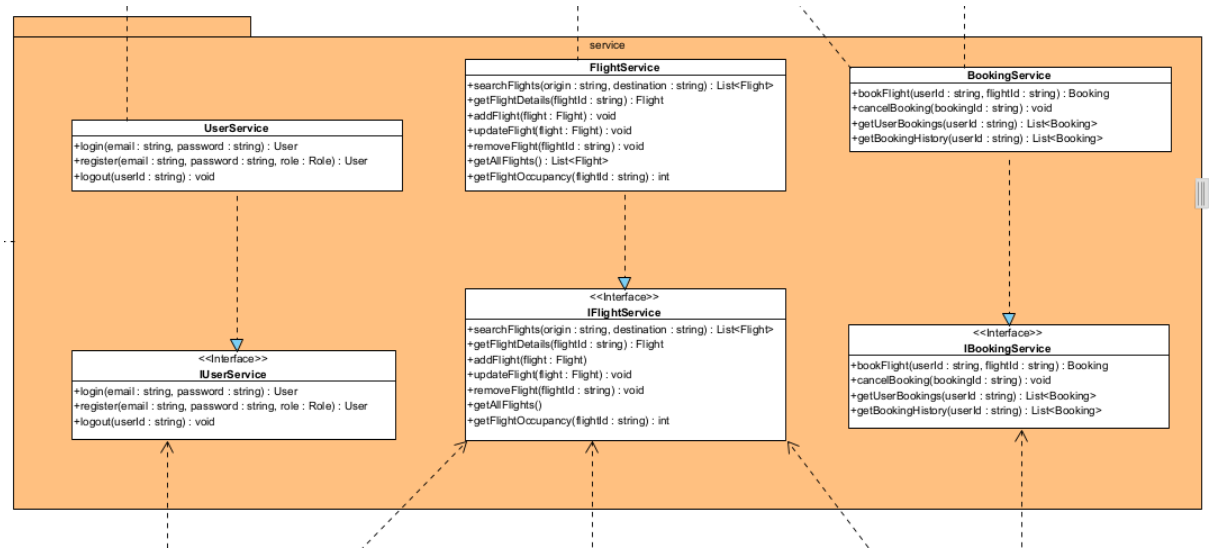
Model Package



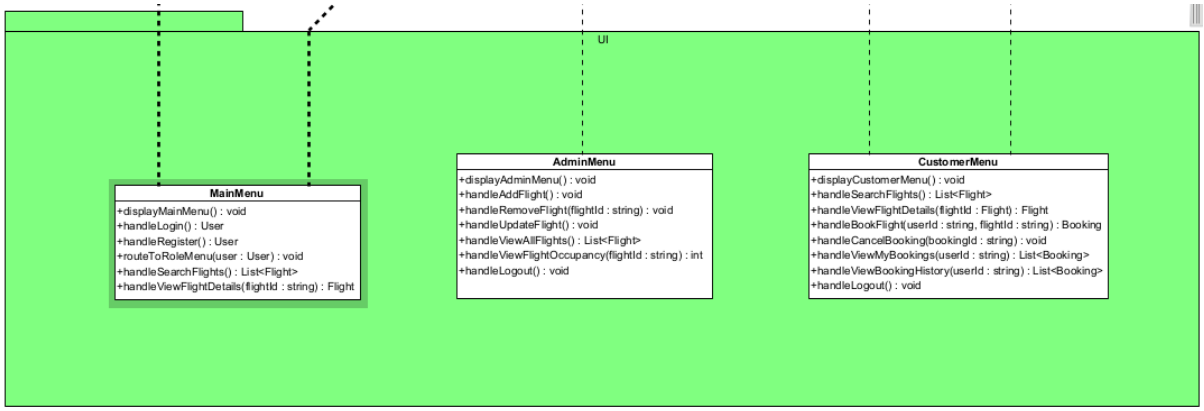
Repository Package



Service Package



UI Package



Mapping between UML classes and planned C# classes

The UML class diagram correspond to C# implementation through four main layers model, repository, service, and UI. In the model layer, the UML classes User, Flight, Booking, together with the enumerations Role and BookingStatus, become the C# domain classes that represents the main business entities of the flight booking system. These classes hold the system's essential data and a few closely related behaviors, such as profile updates for user, seat-related checks for Flight and change status for Booking. In the repository layer, the UML interfaces IUserRepository, IFlightRepository, and IBookingRepository corresponds to C# interfaces while the UserRepository, FlightRepository, Booking Repository maps to their concrete classes where C# implementations sit in. These classes are responsible for data access and data persistence operations such as finding, saving, updating, loading, and deleting records from the chosen data source. The service layer this is where business logic implementation sits in. Interfaces such as IUserService, IFlightService, and IBookingService these classes correspond to its C# implementing classes such as UserService, FlightService, and BookingService. These classes coordinate workflows such as user login, registration, searching and managing flights, and booking or cancelling reservations by calling appropriate repositories and applying validation and business rules. Lastly the UI layer, the UML classes MainMenu, AdminMenu, CustomerMenu map to C# console-based menu or presentation classes that handle user interaction, display menu options, collect input, and call the service layer. The UML to C# mapping shows a layered architecture where each UML class has a clear implementation role from model classes which represent entities, repository classes that manages data access, service classes handle business logic, UI classes manage interaction with the user.

Identification of anticipated design weaknesses prior to OOP application

Prior to OOP application, in the early version of my UML the design initially leans toward to data holders means that domain classes such as User, Flight, and Booking only hold attributes, while most of the behavior is pushed into service classes. This makes the model weak, it violates encapsulation which means to be able to control its own state, and it makes services huge or too many responsibilities which leads to harder to maintain, and test.

Another weakness I've anticipated is no abstraction, no UML interface classes which expose unnecessary details instead of exposing what only is needed. Without abstraction every class handles anything without proper structure which leads to messy and confusing system. Abstraction encourages separation of concerns which helps developers clearly divide responsibilities, it makes the testing easier and improves maintainability.

In the early form of my UML there are customer and admin classes which have an attribute of name, dateOfBirth, address, phoneNumber. There is a base class (user class) which holds the same attributes but has unique attributes UserId, email, and password. I've noticed later that my design lack inheritance which customers and admin hold the same attributes which can result in inconsistent data and violates DRY. Inheritance is important in implementing in any system where it ensures consistency and supports reusability and scalability. If I want to create another object, I can easily inherit attributes from the base class no need to redefine anything and give the system a cleaner design.

So far, those are the design weaknesses I encountered prior to OOP application, the design is still in refinement as this is only the first phase of development. Every design weakness that will be encountered in the next phase will be anticipated and will be resolved.

References:

https://www.airnewzealand.co.nz/executive

https://www.airnewzealand.co.nz/corporate-profile

https://p-airnz.com/cms/assets/PDFs/airnz-2025-annual-report.pdf